Algorytmy 7 + 8 *

Created	@November 19, 2024 2:16 PM
Class	Algorytmika

Struktura danych

Agenda (4h)

1. Pytania przypominające
2. Biblioteka STL
3. Struktura
4. Tablica
5. Lista
6. Kolejka (priorytetowe)
7. Stos
8. Drzewo (Binarne)
9. Sterta
10. Pytania kontrole

Biblioteka STL

1. Biblioteka STL Standard Template Library jest niezwykle pomocnym rozszerzeniem języka Cpp
2. Zawiera wybrane algorytmy i struktury danych udostępnione w postaci szablonów (kod tworzony bez uwzględnienia typów danych)

```
#include <vector>
#include <list>
#include <stack>
#include <queue>
```

Na labach samemu implementujemy struktury danych, w celu lepszego zapoznania się z materiałem

struct - Zgrupowanie ze sobą powiązanych ze sobą informacji, mogą być różnego typu

wskaźniki - Zmienna przechowująca adres pamięci danego elementu

1. Czym jest struktura danych?

Podstawowe definicje

W informatyce struktura danych to organizacja, sposób zarządzania i format przechowywania danych

Struktury danych - podział

Struktura danych może być rozumiana jako pojemnik do przechowywania i organizowania danych. Określa sposób uporządkowania danych na komputerze w taki sposób, aby można było efektywnie uzyskiwać do nich dostęp i aktualizować je

Przykładowo:

Liniowe:

- Statyczne
  - Tablica
- Dynamiczne
  - Lista
  - Stos
  - Kolejka

Nieliniowe:

- Graf
- Drzewo

Oraz:

Homogeniczne:

- Tablica
- Lista

Niehomogeniczne:

- Stos
- Unia
- Struktura

Struktura

Struktura ≠ struktura danych

Jest to niehomogeniczna struktura danych

To typ danych, który pozwala definiować własne typy danych składających się z różnych zmiennych (pól)

Służy do organizowania danych w bardziej przejrzysty sposób.

Wprowadza się ją za pomocą słowa kluczowego struct

Pozwala porządkować przechowywane dane, więc wpisuje się w definicję SD

Z czasem struktury przerodziły się w bardziej zaawansowaną koncepcję (dostępną już w Cpp) - KLASY $ więcej o klasach na programowaniu obiektowym (III sem.)

Struktura - kod w CPP

```
struct Node{
std::string data;
Node* next;//
Node* prev;
```

```
};


Node* newNode = new Node();
rear->next = newNode
newNode->data = "dane"
newNode->prev = rear;
```

Tablica

Podstawowa i pierwsza struktura danych

Umożliwia swobodny dostęp do elementów (za pomocą indeksów)

Elementy są tworzone w pamięci w sposób ciągły

Umożliwia przechowywanie elementów jednego typu (prostego np. int lub złożonego np. struktury)

W C/Cpp numerowanie indeksów rozpoczyna się od 0

String jest tablicą znaków

Możliwe jest stworzenie tablicy wielowymiarowych, wtedy są zapisywane w pamięci wierszami

2. Dlaczego tablica nie jest zawsze najlepszym wyborem podczas tworzenia oprogramowania

Tablica

Zalety:

Upraszcza strukturę programu (zamiast zmiennych t0,t1...,tn mamy tablica[n])

Korzysta z mechanizmu indeksowania (łatwy dostęp do zmiennych)

Wady:

Konieczność rezerwacji pewnego stałego obszaru pamięci

Zmiana wielkości tablicy jest kłopotliwa

Możliwe przekroczenie zakresu tablicy

Lista

Umożliwia przechowywanie jednego typu elementów

Elementy nie znajdują się w jednym bloku pamięci

Dostęp do elementów jest sekwencyjny → dojście do i-tego elementu wymaga przejścia od pierwszego, przez wszystkie pośrednie aż do docelowego elementu

Zwykle oprócz danych, obecny element kolejki będzie przechowywał wskaźnik:

- na element następny (lista jednokierunkowa)
- na element następny i poprzedni (lista dwukierunkowa)

Oprócz elementów listy należy zapamiętać wskaźnik na głowę (pierwszy element listy)

Opcjonalnie można zapamiętać też ogon (ostatni element listy) oraz liczbę elementów

Lista - dodawanie elementów

(Połączenia elementów między listami to wskaźniki)

Aby dodać element:

- Tworzymy nowy element (węzeł) i uzupełniamy go danymi
- Ustawiamy wskaźniki nowego węzł

  Dla dodawania na początek/koniec:

  - Modyfikujemy wskaźnik ostatniego lub pierwszego elementu
  - Modyfikujemy wskaźniki na początek i koniec (head/tail)

  Dla dodawania w środku:

  - Modyfikujemy wskaźniki elementów sąsiadujących z nowym

Aby usunąć element:

- Zmieniamy wskaźniki elementów sąsiadujących, tak by wskazywały jeden na drugi
- Usuwamy element pomiędzy ów elementami

<u>ZAPAMIĘTAĆ NA KOLOKWIUM JAK DODAWAĆ ELEMENTY DO LISTY!!!!!</u>

Lista - dodanie elementu skrajnego

Dodawanie elementu pierwszego:

- Tworzymy nowy element, łączymy go z dawnym pierwszym elementem
- Po połączeniu go, zwiększamy indeksy wszystkich elementów w tablicy o 1, zmieniamy indeks nowego elementu na 0

Dodawanie elementu ostatniego:

- Tworzymy nowy element, łączymy go z dawnym ostatnim elementem
- Po połączeniu go, zmieniamy jego indeks na dawny ostatni indeks + 1

Lista - zastosowanie

Przechowywanie nieuporządkowanych danych - szczególnie jeśli są często usuwanie, dodawane lub edytowanie

Przechowywanie danych o dynamicznym rozmiarze - lista dynamiczna dostosuje rozmiar

Implementacja innych struktur danych (drzewo, grafy)

Obsługa GUI (listy wyboru w formularzach HTML)

Lista - implementacja w STL

std::list

Dynamiczny rozmiar, brak losowego dostępu (dostęp do elementów sekwencyjnych, za pomocą operatorów)

Operacje:

- empty() - zwraca true, jeśli lista jest pusta
- size() - długość listy
- pop_back() - usunięcie z tyłu
- pop_front() - usunięcie z przodu
- begin(), end() - pierwszy i ostatni element bez usuwania go
- insert(), erase() -

Kolejka

Szczególny przypadek listy:

- podobnie jak lista jest homogeniczna
- łatwo zmienia rozmiar
- jej elementy nie muszą być przechowywane blokowo w pamięci

Cechy szczególne:

- jest strukturą danych FIFO (First In, First Out)
- ogranicza dostęp do danych, zezwalając tylko na dodawanie elementów do końca listy i pobieranie ich z początku listy

Kolejka (z priorytetem)

Oprócz danych zawiera informację o ważności elementu (priorytet)

Elementy są dodawane do końca, ale jeśli mają wyższy priorytet niż te w kolejce, “przeskakują je”

Zdejmowanie elementów odbywa się bez zmian

std::priority_queue

Kolejka - zastosowanie

Zarządzanie zadaniami i wątkami w systemach operacyjnych

Kolejki zadań w aplikacjach np. systemy rezerwacyjne

Kolejka drukowania

Routing pakietów w sieci

Obsługa klientów

Kolejka - implementacja w STL

std::queue

Dynamiczny rozmiar, struktura FIFO

Operacje:

- push() - dodanie elementu na koniec kolejki

- pop() - usunięcie elementu z początku kolejki
- front() - pobranie pierwszego elementu bez usuwania go
- back() - pobranie ostatniego elementu bez usuwania go
- empty() - zwraca true, jeśli kolejka jest pusta
- size() - zwraca rozmiar kolejki

Brak losowego dostępu oraz indeksów

Stos

Szczególny przypadek listy:

- podobnie jak lista jest homogeniczny,
- łatwo zmienia rozmiar,
- jego elementy nie muszą być przechowywane blokowo w pamięci.

Cechy szczególne:

- jest strukturą danych LIFO (Last In, First Out)
- ogranicza dostęp do danych, zezwalając tylko na dodawanie i usuwanie elementów z jednego końca listy (szczytu stosu)

Stos - operacje

1. Dodawanie elementu (push)
    a. Stworzenie nowego elementu
    b. Ustawianie wskaźnika na dotychczasowy wierzchołek
    c. Modyfikacja wskaźnika na wierzchołek
2. Zliczanie elementów (dla niepustego stosu)
    a. Rozpocznij od top
    b. Licznik = 1
    c. Dopóki bottom nie wskazuje NULL
        i. Zwiększ licznik i przejdź pod adres bottom
3. Usuwanie elementów (pop)

a. Ustawienie wskaźnika top na bottom dotychczasowego wierzchołka
b. Usunięcie dotychczasowego wierzchołka (delete node)

3. Gdzie możemy spotkać stos?

Rekurencja może przepełnić stos!!

Stos - zastosowanie

- Stos wywołań
- Odwrotna Notacja Polska
- Ctrl+Z - stos jest wykorzystywany

Stos - implementacja w STL

std:stack

Dynamiczny rozmiar, struktura LIFO

Operacje:

- push() - odłożenie elementu na stosie
- pop() -
- top() -
- empty() -
- size() -

Brak losowego dostępu do indeksów - bezpośredni dostęp tylko do wierzchołka stosu (top())

Czym jest odwrotna notacja polska

ONP - RPN - reverse Polish notation

Notacja postfiksowa (operator znajduje się po operandach)

Opracował ją Charles Leonard Hamblin podstawie notacji polskiej (prefiksowej) zaproponowanej przez Jana Łukasiewicza

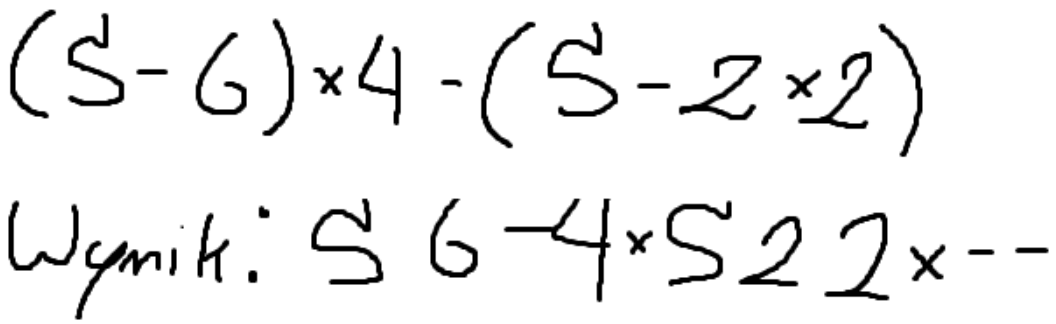
ONP - algorytm konwersji na ONP

1. Stwórz pusty stos operatorów
2. Dla każdego elementu z wyjścia:
    a. Jeśli element jest liczba: podłóż ten element na wyjście
    b. Jeśli elementem jest (: połóż go na stos operatorów
    c. Jeśli elementem jest) : przesuwaj operatory ze stosu na wyjście, dopóki na szczycie stosu nie pojawi się (; wtedy usuń (ze stosu
    d. Jeśli elementem jest operator :) : dopóki na szczycie stosu znajduje się operator o priorytecie wyższym lub równym od priorytetu :), ściągaj operatory ze stosu na wyjście; następnie połóż :) na stosie.
3. Dopóki stos nie jest pusty, ściągaj operatory ze stosu na wyjście

Zadanie 1. Przedstawić za pomocą ONP (Jeżeli na zaliczeniu pojawi się takie zadanie, interesuje ją tylko przedstawienie, a nie wynik obliczenia): (+ schemat blokowy)

Zazwyczaj gwiazdka(*) to operator splotu!! Lecz tutaj jest używana jako mnożenie

(5 - 6) * 4 - (5-2∗2)



ONP - moja wersja

Z liczby normalnej do ONP:

Dokonuj aż skończą się wartości do przeczytania:

1. Wypisujemy normalne liczby do wyniku, od razu gdy je napotkamy
2. Jeżeli spotykamy nawias otwierający, wpisz go do stosu

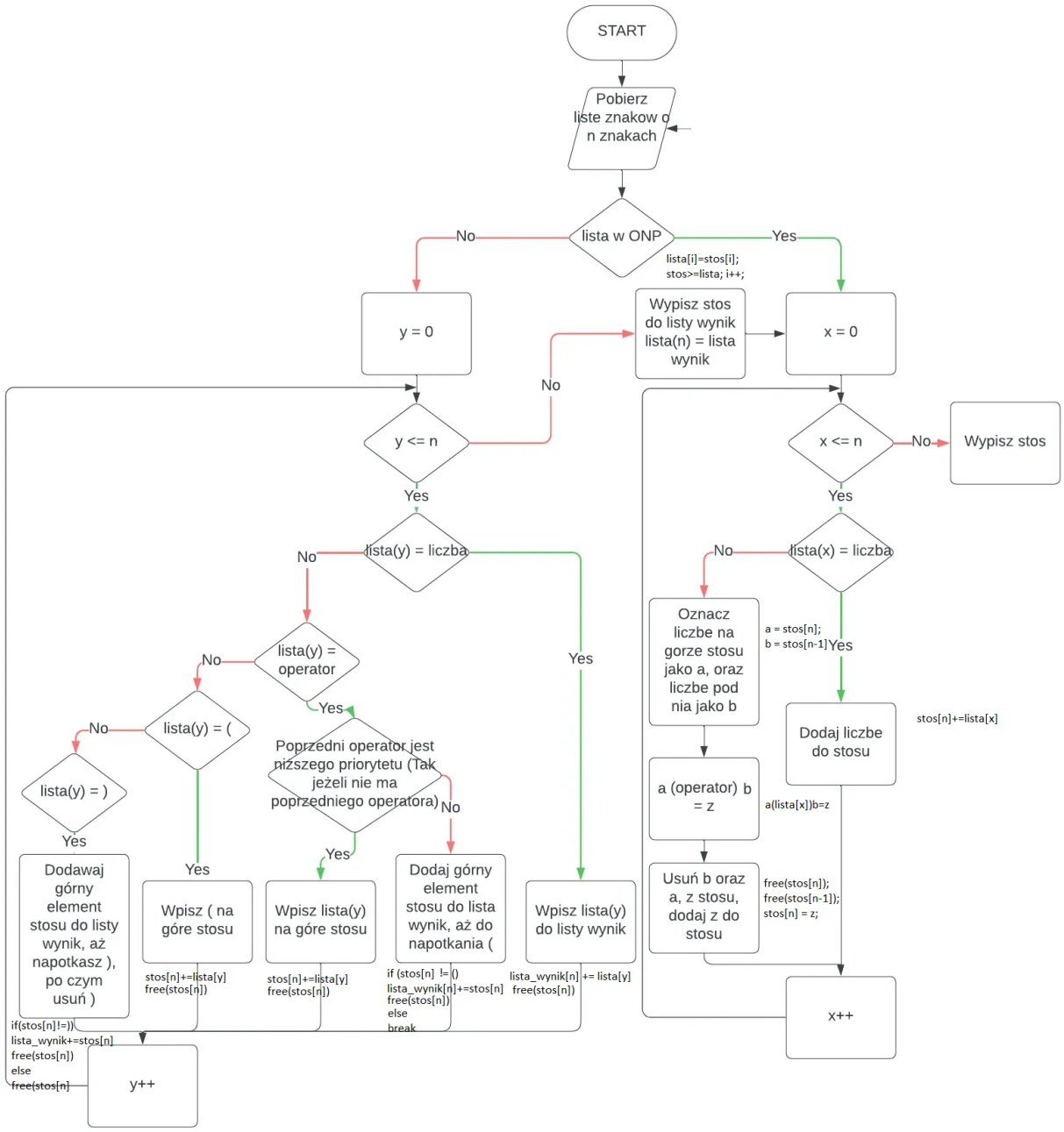
3. Jeżeli napotkamy nawias zamykający, wypisz wszystko z stosu, aż napotkasz nawias otwierający
4. Jeżeli napotykamy operator, idzie on do stosu
    a. Jeżeli operator jest takiego samego stopnia co operator który znajduje się pod nim w stosie, wypisz ten pod nim
    b. Jeżeli operator jest niższego stopnia niż ten pod nim, wypisz cały stos aż napotkasz nawias początkowy
5. Gdy skończą nam się liczby do wypisania, wypisz stos do wyniku

Obliczanie liczby ONP:

Dokonuj aż skończą się wartości do przeczytania:

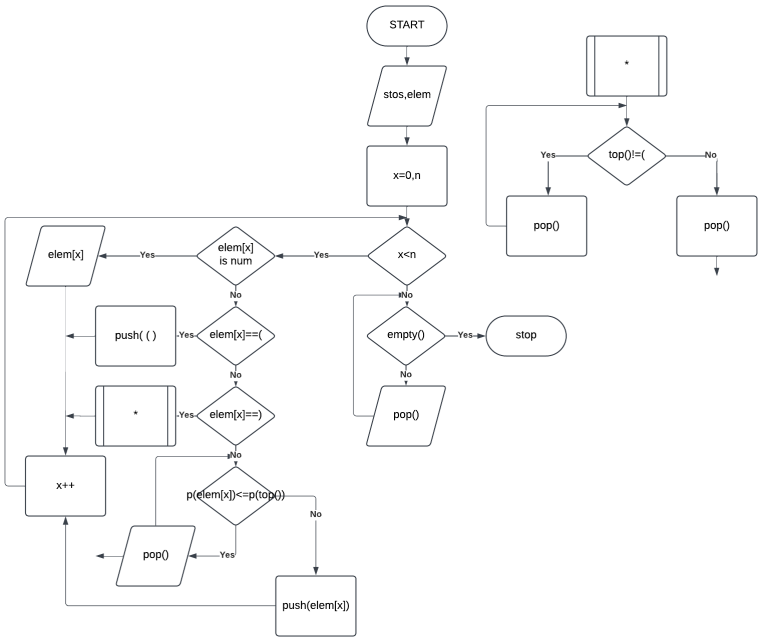
1. Jeżeli spotykamy liczbę, wypisz ją do stosu
2. Jeżeli spotykamy operator, oznacz wartość na górze stosu jako a, a wartość pod nią jako b, usuń te liczby z stosu, i wprowadź wynik z a operator b na góre stosu
3. Jeżeli skończyły się wartości do przeczytania, przenieś stos do wyniku

Schemat blokowy ONP - moja wersja



ONP - Schemat blokowy

<u>ZDEFINIOWAĆ STOS NA POCZĄTKU!!</u>



ONP - algortm obliczenai wartości

1. Stwórz pusty stos.
2. Dla każdego kolejnego elementu z wejścia:
    a. Jeśli elementem jest liczba: połóż ją na stos
    b. Jeśli elementem jest operator :) : ściągnij ze stosu prawy argument b, ściągnij ze stosu lewy argument a, wykonaj obliczenie a :) b, połóż wynik na stos
3. Ściągnij ze stosu i zwróć wynik obliczenia

Drzewo

Hierarchiczna struktura danych

Rozrysowana przypomina drzewo

Składa się z **węzłów** (dane) oraz **krawędzi** (hierarchia)

Pierwszy węzeł to **korzeń**, który ma **węzły podrzędne/dzieci** (oraz jest dla nich **węzłem nadrzędnym/rodzicem**)

Węzeł bez dzieci nazywany jest **liściem**

Węzeł posiadający dzieci będzie nazywany **węzłem wewnętrznym**

Ciąg węzłów połączonych krawędziami to ścieżka, w której liczba krawędzi = dł. ścieżki

Poziom węzła to długość ścieżki od węzła do korzenia (korzeń na poziom 0)

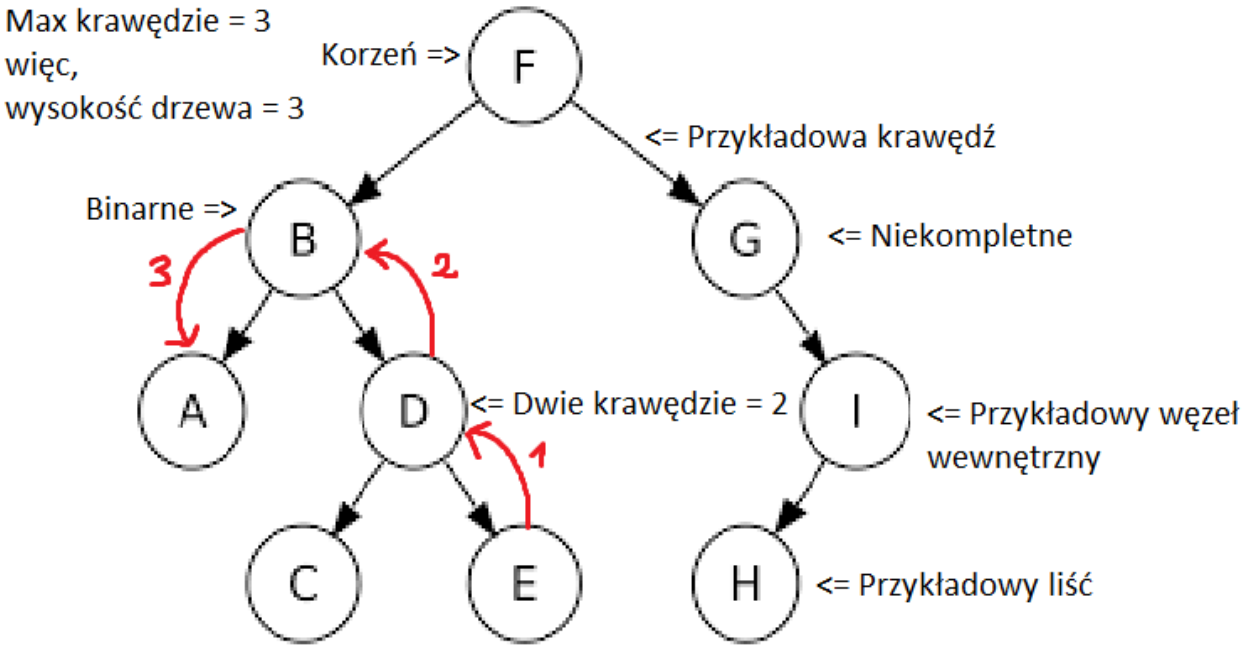
Wysokość drzewa = max. poziom węzła

Jeśli węzeł może mieć max. 2 synów, będzie to **drzewo binarne (B-drzewo)**

Drzewo binarne kompletne ma zapełnione węzłami wszystkie poziomy z wyjątkiem ostatniego, a na ostatnim poziomie węzły są zapełnione począwszy od lewej strony

Zadanie 3. Wskaż:

- Węzły, korzeń, liście, węzły wewnętrzne
- Krawędzie
- Ścieżkę E-A i jej długość
- Poziom węzła D
- Wysokość drzewa
- B-Drzewo?
- Kompletne?



Sterta (in. stóg, kopiec)

Szczególny typ struktury drzewiastej

Jest to kompletne drzewo binarne

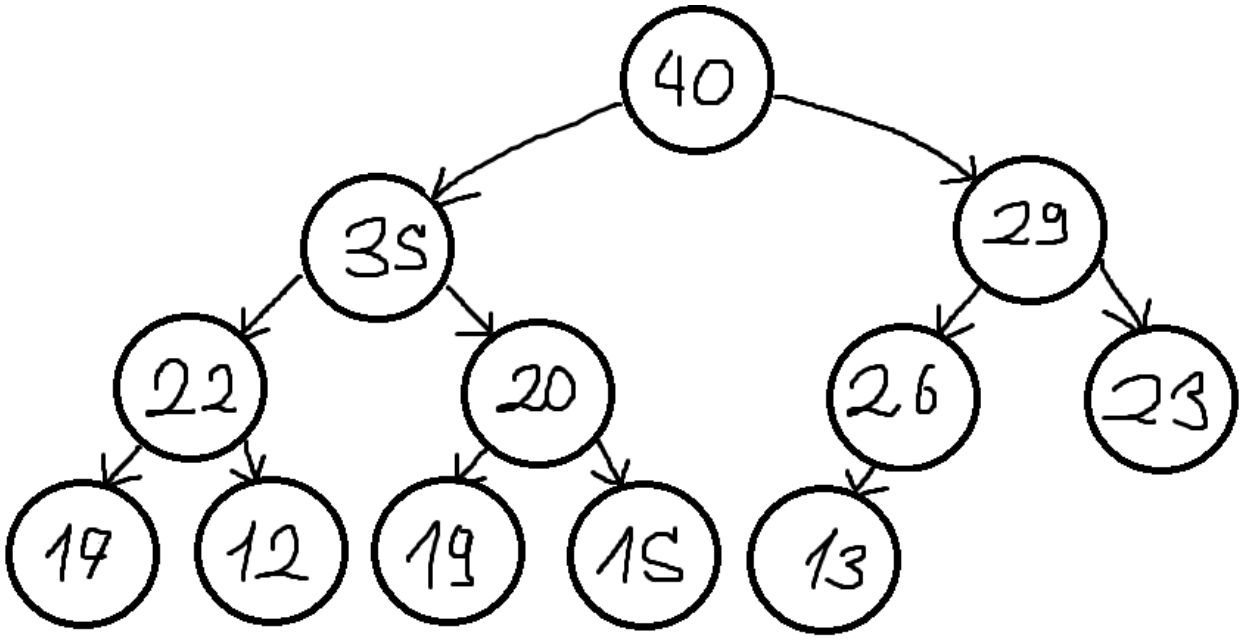
Choć zbiór danych w stercie nie jest uporządkowany globalnie, ma fragmenty lokalnie uporządkowane

Uporządkowanie: wartość każdego z dwóch jego potomków jest nie większa od wartości ojca (dla sterty MAX) - warunek “niemniejszy” dla sterty MIN

Zastosowanie: kolejka priorytetowa, sortowanie liczb (Heapsort) - zwykle tylko 2x wolniejsze od Quicksorta

Zadanie 4. Dodajemy do sterty elementy:

35 40 29 17 19 13 23 22 12 20 15 26



Sterta - usuwanie elementów

1. Usuń korzeń
2. Zastąp go ostatnim elementem w stercie
3. Porównaj korzeń z jego synami. Jeśli trzeba, zamień korzeń z większym z synów

4. Wykonuj porównanie i zamianę aż do dołu drzewa lub znalezienia odpowiedniego miejsca

Binarne drzewo poszukiwań (Binary Search Tree, BST)

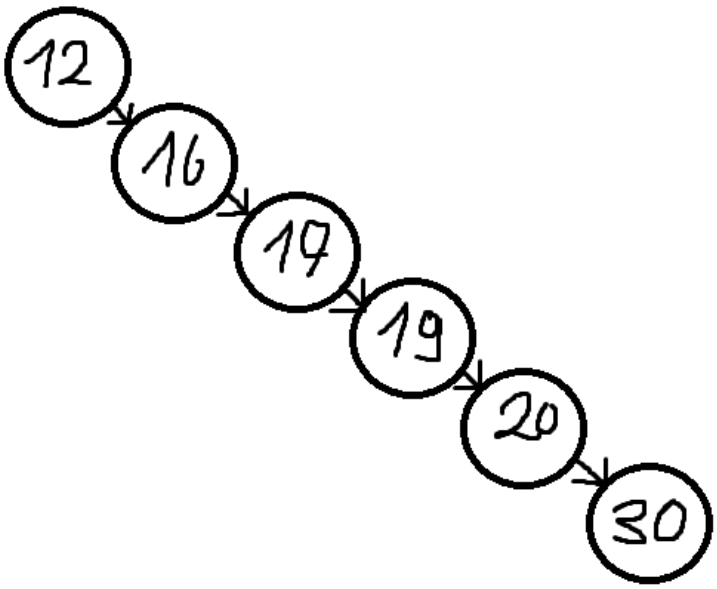
- Lewy potomek węzła zawiera wartość mniejszą od węzła
- Prawy potomek węzła zawiera wartość większą od węzła
- Prosty do zrozuzmienia, ale wymaga równoważenia, aby utrzymać efektywność (modyfikacje: AVL, d. czerwono-czarne)

Dodawanie elementu do drzewa BST:

1. Niech korzeń będzie wierzchołkiem aktualnym
2. Jeśli wartość nowego elementu ≤ wartość aktualnego wierzchołka, przejdź do lewego syna. W przeciwnym wypadku przejdź do prawego syna
3. Powtarzaj krok 2 dopóki nie dojdziesz do liścia
4. Wstaw nowy wierzchołek jako prawego lub lewego syna znalezionego liścia w zależności od ich wartości

Zadanie 6. Dodaj do BST

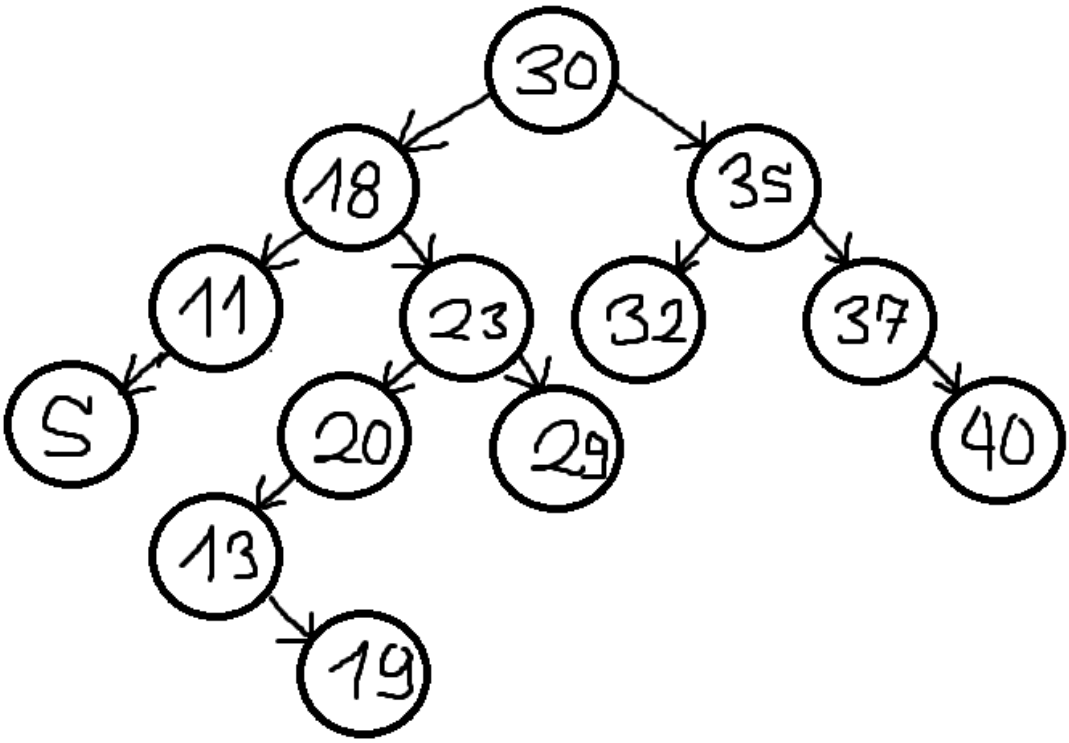
12 16 17 19 20 30



BST nie ma mechanizmu równoważenia, przez co korzystanie z niego. dla zbiorów posortowanych mija się z celem (tworzymy listę)

Zadanie 7. Dodajemy do BST:

30 18 23 35 37 32 11 5 20 29 13 19 40

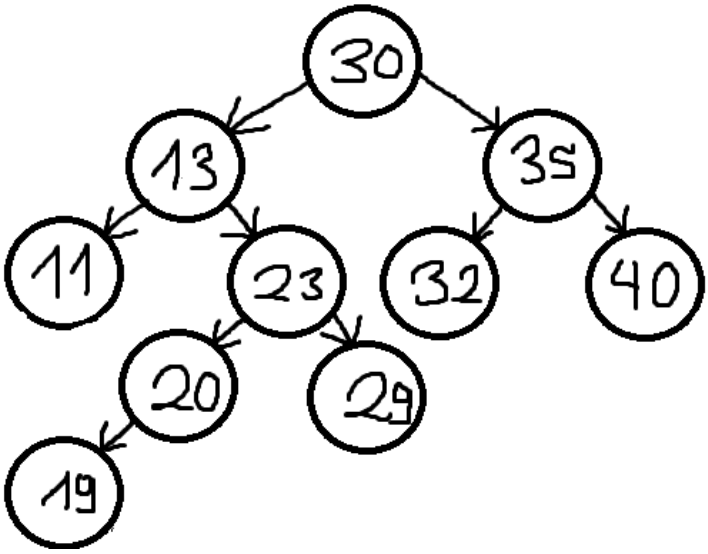


Usuwanie elementów z BST:

1. Węzeł jest liściem - Po prostu usuń węzeł, ustawiając wskaźnik w jego rodzicu na NULL
2. Węzeł ma jedno dziecko - Zastęp usuwany węzeł jego jedynym dzieckiem (czyli wskaźnik rodzica węzła przestaw na dziecko usuwanego węzła)
3. Węzeł ma dwoje dzieci - znajdź następnika in-order usuwanego węzła (najmniejszy element w prawym poddrzewie). Skopiuj wartość następnika in-order do usuwanego węzła. Usuń następnik in-order, stosując powyższe reguły (liść lub jedno dziecko)

Przeszukiwanie drzewa BST jest analogiczne do algorytmu binary search

Zadanie 8. Usuńmy elementy 5, 37 i 18



Drzewo czerwono-czarne i AVL (Adelson-Velsky-Landis)

- Wprowadzone, aby zachować balans w drzewie
- Korzystają z mechanizmu rotacji
- **Drzewo czerwono czarne ma 2 razy mniej rotacji niż AVL!!! Ponieważ jest dokładne tylko do 1 stopnia!!**

BST vs Drzewo Czerwono-Czarne vs Drzewo AVL

Cecha	BST	Drzewo AVL	Drzewo Czerwono-Czarne
Złożoność wyszukiwania	Średnio **O(log n)** Pesymistycznie **O(n)**	**O(log n)**	**O(log n)**
Złożoność wstawiania	Średnio **O(log n)** Pesymistycznie **O(n)**	**O(log n)**, z dodatkowymi rotacjami	**O(log n)**, mniej rotacji niż AVL
Złożoność usuwania	Średnio **O(log n)** Pesymistycznie **O(n)**	**O(log n)**, z dodatkowymi rotacjami	**O(log n)**, mniej rotacji niż AVL
Równowaga drzewa	Brak	Wysokości poddrzew dowolnego węzła różnią się max o 1	Wysokości poddrzew dowolnego węzła mogą się różnić do ok. 2 razy
Częstość rotacji	Brak	Częste	Rzadsze niż w AVL

Zbiór

Def.(ang. Set) Kontener, który przechowuje unikalne elementy w nieokreślonej kolejności

Cechy:

- Nie dopuszcza duplikatów (każdy element występuje tylko raz)
- Elementy są automatycznie sortowane (w przypadku std::set)

Operacje:

- Dodawanie elementu
- Usuwanie elementu

Słownik:

Def. (ang. Map), to kontener par klucz-wartość, gdzie każdy klucz jest unikalny

Cechy:

- Każdy klucz jest przypisany do jednej wartości
- Elementy są automatycznie sortowane według klucza (w przypadku std::map)

Operacje:

- Dodawanie elementu
- Usuwanie elementu
- Wyszukiwanie wartości po kluczu
- Dostęp do wartości

Przykład zastosowania:

- Przechowywanie konfiguracji

Wektor:

Def. (ang. Vector) dynamiczna tablica, która może zmienić rozmiar w trakcie działania programu

Cechy:

- Elementy przechowywane są w sposób ciągły w pamięci
- Rozmiar wektora rośnie dynamicznie, gdy jest taka potrzeba

BST vs AVL vs Drzewo Czerwono-Czarne

Drzewo BST działa dobrze dla losowych danych, może się zdegradować do listy w przypadku uporządkowanych danych

Średnio wyszukiwanie elementów w drzewie, wynosi O(log n), lecz jeżeli drzewo BST zostanie zdegradowane do listy, złożoność wzrasta do O(n)

Tak samo jest ze złożonością wstawiania i usuwania, średnio wychodzi O(log n), lecz podstawowe drzewo BST może być zdegradowane do O(n)

Równowaga drzewa nie istnieje w drzewie BST, lecz w drzewie AVL, różnica wysokości poddrzew wynosi maksymalnie 1. W Drzewie Czerwono-Czarnym, ta różnica może wynosić 2

Drzewo AVL, gwarantuje równowagę dlatego, średni czas wykonywania jest najmniejszy, najlepsze w wyszukiwaniu elementów

Drzewo Czerwono-Czarne, jest najlepsze do wstawiania/usuwania

Pytania kontrolne

1. Czym jest struktura danych?

- Jest to organizacja, sposób zarządzania i format przechowywania danych

2. Czym jest struktura? Jakim słowem kluczowym w Cpp ją definiujemy?

- Definiowana przez ‘struct’, jest to niehomogeniczna struktura danych, pozwalająca na definiowanie własnych typów danych składających się z różnych zmiennych

3. Tablica jako struktura danych - opis, wady, zalety

- Homogeniczna struktura danych, pozwalająca na zapiyswanie w niej tylko jednego typu wartości. Jej zaletami są uproszczenie struktury programu i indeksy, jej wadami są min: konieczność rezerwacji pamięci w momencie jej utworzenia, trudności w zmianie jej rozmiaru oraz możliwość przekroczenia jej granicy

4. Jak dodaje się i usuwa elementy z listy?

- By dodać element, musimy zmienić indeksy elementów w liście, tak by elementy znajdujące się po nowo wprowadzonym elemencie, miały indeks większe o 1, po czym wstawiamy nasz nowy węzeł w odpowiedni indeks, wypełniamy go danymi, i łączymy wskaźniki sąsiadujących elementów do

nowego węzła, tak by kierowały na następny i poprzedni w odpowiedni sposób. By usunąć element, musimy pierw zmienić wskaźniki elementów sąsiadujących w taki sposób by wskazywały na siebie, po czym usunąć nasz węzeł, i zmienić indeksy elementów po nim o -1

5. Cechy wspólne i wyróżniające - kolejka vs. stos vs. lista

- Wszystkie trzy są typami listy, przechowują jeden typ danych, ich elementy nie muszą być przechowywane blokowo w pamięci. Kolejka różni się tym, że jest strukturą danych First In, First out, ogranicza dostęp do danych, jak i pozwala tylko na dodawanie elementów do końca listy, i pobierania ich z początku, jak i posiada możliwość wykorzystania priorytetu. Stos różni się tym, że jest strukturą danych Last In, First Out, zezwala on na dodawanie i odejmowanie elementów tylko z jego końca

6. Jakie są realistyczne zastosowania stosu i kolejki?

- Przykładowymi zastosowaniami stosu jest między innymi: stos wywołań, odwrotna notacja polska jak i akcja ctrl+z. Przykładowymi zastosowaniami kolejki są między innymi: zarządzanie zadaniami i wątkami w systemach operacyjnych, kolejki zadań w aplikacji, routing pakietów jak i kolejka drukowania

7. Czym jest ONP?

- Odwrócona Notacja Polska, to notacja postfiksowa, czyli zapis operatorów PO liczbach

8. Jakie zmiany wprowadza kolejka priorytetowa?

- Dodaje informację o ważności elementu, czyli wartość priorytetu, tak samo jak w liście, elementy są dodawane do końca, lecz jeżeli mają wyższy priorytet od elementów przed nim, zmieniają miejsce aż napotkają element z wyższym priorytetem, lub zajmą pierwsze miejsce

9. Do realizacji których elementów algorytmów ONP wykorzystamy stos?

- Stos jest wykorzystywany do przetrzymywania operatorów, przy zamianie na notacje postfiksową, lub do obliczania wartości liczby, przy obliczaniu wartości zapisanej w notacji postfiksowej

10. Jakie są podstawowe pojęcia związane z drzewami?

- Węzeł - dane (wpisane w kołach przy zapisie graficznym).
  Krawędź - połączenie dwóch węzłów, oraz ich hierarchia.
  Korzeń - pierwszy węzeł.
  Dzieci - węzły wychodzące z węzła o wyższej hierarchi.
  Rodzic - węzeł z którego wychodzą inne węzły.
  Liść - węzeł bez dzieci.
  Węzeł wewnętrzny - węzeł posiadający dzieci.
  Ścieżka - ciąg węzłów połączonych krawędziami, jej długość to ilość krawędzi pomiędzy dwoma elementami.
  Poziom węzła - długość ścieżki od węzła do korzenia.
  Wysokość drzewa - maksymalny poziom węzła.
  Drzewo binarne - drzewo w którym każdy węzeł może mieć maksymalnie 2 dzieci.
  Drzewo binarne kompletne - drzewo binarne których wszystkie węzły poza ostatnimi mają po 2 dzieci, na ostatnim poziomie węzły są zapełnione od lewej strony do prawej

11. Czym jest sterta? Jak wygląda dodawanie/usuwanie elementów z niej? Czym jest MAX/MIN sterta?

- Stera, inaczej stóg lub kopiec, to specjalny typ struktury drzewiastej. Jest to kompletne drzewo binarne, uporządkowane lokalnie.
  Dodajemy elementy do najwyższego węzła po lewej stronie, aż będzie posiadał 2 dzieci, jeżeli nowo dodany element jest większy od rodzica, dochodzi do zamiany miejsc aż odpowiednia kolejność zostanie odzyskana.
  Usuwamy elementy z sterty poprzez usunięcie korzenia, i zastąpienie go najnowszym elementem w stercie, i w razie potrzeby zamieniami go z większymi synami.
  Sterta MAX, to sterta w którym dzieci nie mogą być większe od rodzica.
  Sterta MIN, to sterta w którym rodzic musi być niemniejszy od dzieci

12. Czym jest drzewo BST? Jak wskazać MIN/MAX w drzewie BST? Jakie modyfikacje BST zostały omówione? (Złożoność obliczeniowa, co wykorzystują)

- Drzewo BST to specjalny typ drzewa binarnego, w którym elementy większe od rodzica znajdują się na prawo od niego, a mniejsze na lewo.
  MIN znajdujemy poprzez przechodzenie zawsze poprzez lewą krawędź, aż do

napotkania liścia. MAX znajdujemy poprzez przechodzenie zawsze poprzez prawą krawędź, aż do napotkania liścia.
Omówione modyfikacje do drzewa BST to: AVL oraz Drzewo Czerwono-Czarne, które wykorzystują mechanizm rotacji do zachowania balansu.
Drzewo BST nie posiada mechanizmu zachowującego balans, więc jego średnie wyszukiwanie i wstawianie/usuwanie O(log n), może być zdegradowane do listy O(n).
AVL posiada mechanizm zachowujący balans, który pozwala na maksymalną różnice podrzew wynoszącą 1. Drzewo AVL, gwarantuję prawie perfekcyjną równagę, dlatego jego średni czas wyszukiwania wynosi O(log n).
Drzewo Czerwono-Czarne, podobnie do AVL, posiada mechanizm zachowujący balans, który pozwala na maksymalną różnice podrzew wynoszącą 2. Drzewo Czerwono-Czarne, nie gwarantuję aż takiej wysokiej równowagi co AVL, lecz jest efektywniejsze w wstawianiu i usuwaniu elementów

13. Co wyróżnia strukturę danych zbiór?

- Zbiór to kontener, który przechowuje unikalne elementy w nieokreślonej kolejności, nie dopuszcza on duplikatów, w przypadku std::set, jego elementy są automatycznie sortowane

14. Co wyróżnia strukturę danych słownik?

- Słownik to kontener par klucz-wartość, gdzie każdy klucz jest unikalny i przypisany do jednej wartości, w przypadku std::map, jego elementy są automatycznie sortowane według klucza

15. Co wyróżnia strukturę danych wektor?

- Wektor to dynamiczna tablica, która może zmieniać rozmiar w trakcie działania programu